

Invariant Generation

It is easier to write an incorrect program than [to] understand a correct one.

— Alan Perlis

Epigrams on Programming, 1982

While applying the inductive assertion method of Chapters 5 and 6 certainly requires insights from the programmer, algorithms that examine programs can discover many simple properties. This chapter describes a form of **static analysis** called **invariant generation**, whose task is to discover inductive assertions of programs. A static analysis is a procedure that operates on the text of a program. An invariant generation procedure is a static analysis that produces inductive program annotations as output.

Section 12.1 discusses the general context of invariant generation and describes a methodology for constructing invariant generation procedures. Applying this methodology, Section 12.2 describes **interval analysis**, an invariant generation procedure that discovers invariants of the form $c \leq v$ or $v \leq c$, for program variable v and constant c . Section 12.3 describes **Karr's analysis**, an invariant generation procedure that discovers invariants of the form $c_0 + c_1x_1 + \cdots + c_nx_n = 0$, for program variables $x_1, \dots, x_n$ and constants $c_0, c_1, \dots, c_n$.

Many other invariant generation algorithms are studied in the literature, including analyses to discover linear inequalities $c_0 + c_1x_1 + \cdots + c_nx_n \leq 0$, polynomial equalities and inequalities, and facts about memory and variable aliasing. **Bibliographic Remarks** refers the interested reader to example papers on these topics.

12.1 Invariant Generation

After revisiting the **weakest precondition** and defining the **strongest postcondition** in Sections 12.1.1 and 12.1.2, Section 12.1.3 describes the general

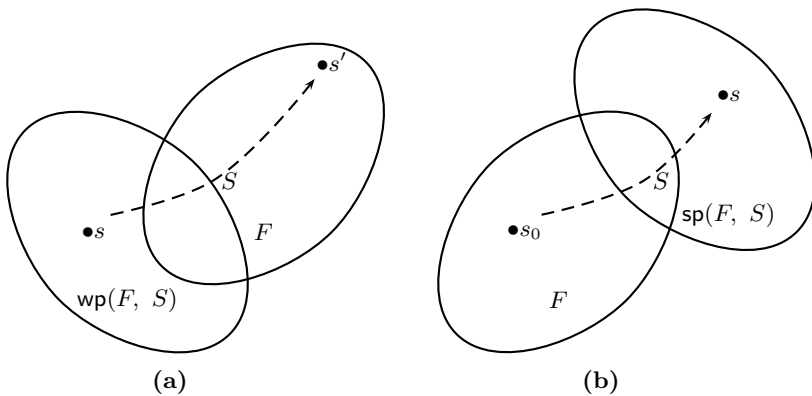


Fig. 12.1. (a) Weakest precondition and (b) strongest postcondition

forward propagation procedure for discovering inductive assertions. In general, this procedure is not an algorithm: it can run forever without producing an answer. Hence, Section 12.1.4 presents a methodology for constructing **abstract interpretations** of programs, which focus on particular elements of programs and apply a heuristic to guarantee termination. Subsequent sections examine particular instances of this methodology.

12.1.1 Weakest Precondition and Strongest Postcondition

Recall from Section 5.2.4 that a **predicate transformer** p is a function

$$p : \text{FOL} \times \text{stmts} \rightarrow \text{FOL}$$

that maps a FOL formula $F \in \text{FOL}$ and program statement $S \in \text{stmts}$ to a FOL formula. The weakest precondition predicate transformer $\text{wp}(F, S)$ is defined on the two types of program statements that occur in basic paths:

- $\text{wp}(F, \text{assume } c) \Leftrightarrow c \rightarrow F$, and
- $\text{wp}(F[v], v := e) \Leftrightarrow F[e]$;

and inductively on sequences of statements $S_1; \dots; S_n$:

$$\text{wp}(F, S_1; \dots; S_n) \Leftrightarrow \text{wp}(\text{wp}(F, S_n), S_1; \dots; S_{n-1}) .$$

The weakest precondition $\text{wp}(F, S)$ has the defining characteristic that if state s is such that

$$s \models \text{wp}(F, S)$$

and if statement S is executed on state s to produce state s' , then

$$s' \models F .$$

In other words, the weakest precondition moves a formula backward over a sequence of statements: for F to hold after executing $S_1; \dots; S_n$, the formula $\text{wp}(F, S_1; \dots; S_n)$ must hold before executing the statements.

This situation is visualized in Figure 12.1(a). The region labeled F is the set of states that satisfy F ; similarly, the region labeled $\text{wp}(F, S)$ is the set of states that satisfy $\text{wp}(F, S)$. Every state s on which executing statement S leads to a state s' in the F region must be in the $\text{wp}(F, S)$ region.

For reasoning in the opposite direction, we define the **strongest postcondition** predicate transformer. The strongest postcondition $\text{sp}(F, s)$ has the defining characteristic that if s is the current state and

$$s \models \text{sp}(F, S)$$

then there exists a state s_0 such that executing S on s_0 results in state s and

$$s_0 \models F.$$

Figure 12.1(b) visualizes this characteristic. Executing statement S on any state s_0 in the F region must result in a state s in the $\text{sp}(F, S)$ region.

Define $\text{sp}(F, S)$ as follows. On **assume** statements,

$$\text{sp}(F, \text{assume } c) \Leftrightarrow c \wedge F,$$

for if program control makes it past the statement, then c must hold.

Unlike in the case of wp , there is no simple definition of sp on assignments:

$$\text{sp}(F[v], v := e[v]) \Leftrightarrow \exists v^0. v = e[v^0] \wedge F[v^0].$$

Let s_0 and s be the states before and after executing the assignment, respectively. v^0 represents the value of v in state s_0 . Every variable other than v maintains its value from s_0 in s . Then $v = e[v^0]$ asserts that the value of v in state s is equal to the value of e in state s_0 . $F[v^0]$ asserts that $s_0 \models F$. Overall, $\text{sp}(F, v := e)$ describes the states that can be obtained by executing $v := e$ from F -states, states that satisfy F .

Finally, define sp inductively on a sequence of statements $S_1; \dots; S_n$:

$$\text{sp}(F, S_1; \dots; S_n) \Leftrightarrow \text{sp}(\text{sp}(F, S_1), S_2; \dots; S_n).$$

sp progresses forward through the statements.

Example 12.1. Compute

$$\begin{aligned} \text{sp}(i \geq n, i := i + k) \\ \Leftrightarrow \exists i^0. i = i^0 + k \wedge i^0 \geq n \\ \Leftrightarrow i - k \geq n \end{aligned}$$

since $i^0 = i - k$.

Compute

$$\begin{aligned}
& \text{sp}(i \geq n, \text{assume } k \geq 0; i := i + k) \\
& \Leftrightarrow \text{sp}(\text{sp}(i \geq n, \text{assume } k \geq 0), i := i + k) \\
& \Leftrightarrow \text{sp}(k \geq 0 \wedge i \geq n, i := i + k) \\
& \Leftrightarrow \exists i^0. i = i^0 + k \wedge k \geq 0 \wedge i^0 \geq n \\
& \Leftrightarrow k \geq 0 \wedge i - k \geq n
\end{aligned}$$

■

Example 12.2. Let us prove that

$$\text{sp}(\text{wp}(F, S), S) \Rightarrow F \Rightarrow \text{wp}(\text{sp}(F, S), S);$$

that is, the strongest postcondition of the weakest precondition of F on statement S implies F , which implies the weakest precondition of the strongest postcondition of F on S . We prove the first implication and leave the second implication as Exercise 12.1.

Suppose that S is the statement **assume** c . Then

$$\begin{aligned}
& \text{sp}(\text{wp}(F, \text{assume } c), \text{assume } c) \\
& \Leftrightarrow \text{sp}(c \rightarrow F, \text{assume } c) \\
& \Leftrightarrow c \wedge (c \rightarrow F) \\
& \Leftrightarrow c \wedge F \\
& \Rightarrow F
\end{aligned}$$

Now suppose that S is an assignment statement $v := e$. Then

$$\begin{aligned}
& \text{sp}(\text{wp}(F[v], v := e[v]), v := e[v]) \\
& \Leftrightarrow \text{sp}(F[e[v]], v := e[v]) \\
& \Leftrightarrow \exists v^0. v = e[v^0] \wedge F[e[v^0]] \\
& \Leftrightarrow \exists v^0. v = e[v^0] \wedge F[v] \\
& \Rightarrow F
\end{aligned}$$

■

Recall the definition of a verification condition in terms of **wp**:

$$\{F\}S_1; \dots; S_n\{G\} : F \Rightarrow \text{wp}(G, S_1; \dots; S_n).$$

We can similarly define a verification condition in terms of **sp**:

$$\{F\}S_1; \dots; S_n\{G\} : \text{sp}(F, S_1; \dots; S_n) \Rightarrow G.$$

Typically, we prefer working with the weakest precondition because of its syntactic handling of assignment statements. However, in the remainder of this chapter, we shall see the value of the strongest postcondition.

12.1.2 ★General Definitions of wp and sp

Section 12.1.1 defines the **wp** and **sp** predicate transformers for our simple language of assumption (**assume** c) and assignment ($v := e$) statements. This section defines these predicate transformers more generally.

Describe program statements with FOL formulae over the program variables $\bar{x}$, the **program counter** pc , and the **primed variables** $\bar{x}'$ and pc' . The program counter ranges over the locations of the program. The primed variables represent the values of the corresponding unprimed variables in the next state. For example, the statement

$L_i : \text{assume } c;$
 $L_j :$

is captured by the relation

$$\rho[pc, \bar{x}, pc', \bar{x}'] : pc = L_i \wedge pc' = L_j \wedge c \wedge \bar{x}' = \bar{x} . \quad (12.1)$$

To execute this statement, the program counter must be L_i and c must be **true**. Executing the statement sets the program counter to L_j and leaves the variables $\bar{x}$ unchanged. Similarly, the assignment

$L_i : x_i := e;$
 $L_j :$

is captured by the relation

$$\rho[pc, \bar{x}, pc', \bar{x}'] : pc = L_i \wedge pc' = L_j \wedge x'_i = e \wedge \text{pres}(\bar{x} \setminus \{x_i\}) , \quad (12.2)$$

where $\text{pres}(V)$, for a set of variables V , abbreviates the assertion that each variable in V remains unchanged:

$$\bigwedge_{v \in V} v' = v .$$

Formulae (12.1) and (12.2) are called **transition relations**. The expressiveness of FOL allows many more constructs to be encoded as transition relations than can be encoded in either **pi** or the simple language of assumptions and assignments.

Let us consider the **weakest precondition** and the **strongest postcondition** in this general context. For convenience, let $\bar{y}$ be all the variables of the program including the program counter pc . The weakest precondition of F over transition relation $\rho[\bar{y}, \bar{y}']$ is given by

$$\text{wp}(F, \rho[\bar{y}, \bar{y}']) \Leftrightarrow \forall \bar{y}'. \rho[\bar{y}, \bar{y}'] \rightarrow F[\bar{y}'] .$$

Notice that $\text{free}(\text{wp}(F, \rho)) = \bar{y}$. A satisfying $\bar{y}$ represents a state from which all ρ -successors, which are states $\bar{y}'$ that ρ relates to $\bar{y}$, are F -states. Technically,

such a state might not have any successors at all; for example, ρ could describe a guard that is **false** in the state described by $\overline{y}$.

Consider the transition relation (12.1) corresponding to **assume** c :

$$\begin{aligned} \text{wp}(F, pc = L_i \wedge pc' = L_j \wedge c \wedge \overline{x}' = \overline{x}) \\ \Leftrightarrow \forall pc', \overline{x}'. pc = L_i \wedge pc' = L_j \wedge c \wedge \overline{x}' = \overline{x} \rightarrow F[pc', \overline{x}'] \\ \Leftrightarrow pc = L_i \wedge c \rightarrow F[L_j, \overline{x}] \end{aligned}$$

Disregarding the program counter, we have recaptured our original definition of **wp** on **assume** statements:

$$\text{wp}(F, \text{assume } c) \Leftrightarrow c \rightarrow F .$$

Similarly,

$$\begin{aligned} \text{wp}(F, pc = L_i \wedge pc' = L_j \wedge x'_i = e \wedge \text{pres}(\overline{x} \setminus \{x_i\})) \\ \Leftrightarrow \forall pc', \overline{x}'. pc = L_i \wedge pc' = L_j \wedge x'_i = e \wedge \text{pres}(\overline{x} \setminus \{x_i\}) \\ \rightarrow F[pc', x'_1, \dots, x'_i, \dots, x'_n] \\ \Leftrightarrow pc = L_i \rightarrow F[L_j, x_1, \dots, e, \dots, x_n] \end{aligned}$$

Again, disregarding the program counter reveals our original definition:

$$\text{wp}(F[v], v := e) \Leftrightarrow F[e] .$$

The general form of the strongest postcondition is the following:

$$\text{sp}(F, \rho) \Leftrightarrow \exists \overline{y}_0. \rho[\overline{y}_0, \overline{y}] \wedge F[\overline{y}_0] .$$

Notice that $\text{free}(\text{sp}(F, \rho)) = \overline{y}$. It describes all states $\overline{y}$ that have some ρ -predecessor $\overline{y}_0$ that is an F -state; or, in other words, it describes all ρ -successors of F -states.

Exercise 12.2 asks the reader to specialize this definition of **sp** to the case of assumption and assignment statements. Exercise 12.3 asks the reader to reproduce the arguments of Example 12.2 and Exercise 12.1 in this more general setting.

12.1.3 Static Analysis

We now turn to the task of generating inductive information about programs. Throughout this chapter, we consider programs with just a single function. Exercise 12.6 asks the reader to generalize the methods to treat programs with multiple, possibly recursive, functions.

Consider a function with locations $\mathcal{L}$ forming a cutset including the initial location L_0 . A **cutset** is a set of locations such that every path that begins and ends at a pair of **cutpoints** — a location in the cutset — without including another cutpoint is a basic path. An **assertion map**

$$\mu : \mathcal{L} \rightarrow \text{FOL}$$

is a map from the set $\mathcal{L}$ to first-order assertions. Assertion map μ is an **inductive assertion map**, also called an **inductive map**, if for each basic path

$$\begin{aligned} L_i &: @ \mu(L_i) \\ S_i; \\ &\vdots \\ S_j; \\ L_j &: @ \mu(L_j) \end{aligned}$$

for $L_i, L_j \in \mathcal{L}$, the verification condition

$$\{\mu(L_i)\}S_i; \dots; S_j\{\mu(L_j)\} \quad (12.3)$$

is valid. The task of invariant generation is to find inductive assertion maps μ . Viewing each $\mu(L_i)$ as a variable that ranges over FOL formulae, the task of invariant generation is to find solutions to the constraints imposed by the verification conditions (12.3) for all basic paths. To avoid trivial solutions, we require that at function entry location L_0 with function precondition F_{pre} , $F_{\text{pre}} \Rightarrow \mu(L_0)$.

How can we solve constraints (12.3) for μ ? One common technique is to perform a **symbolic execution** of the function using the strongest postcondition predicate transformer. This process is also called **forward propagation** because information flows forward through the function. The idea is to represent sets of states by formulae over the program variables. **sp** provides a mechanism for executing the function over formulae, and thus over sets of states.

Suppose that the function has function precondition F_{pre} at initial location L_0 . Define the initial assertion map μ : let

$$\mu(L_0) := F_{\text{pre}} \text{ , and for } L \in \mathcal{L} \setminus \{L_0\}, \quad \mu(L) := \perp \text{ .}$$

This configuration represents entering the function with some values satisfying its precondition.

Maintain a set $S \subseteq \mathcal{L}$ of locations that still need processing. Initially, let $S = \{L_0\}$. Terminate when $S = \emptyset$.

Suppose that we are on iteration i , having constructed the intermediate assertion map μ . Choose some location $L_j \in S$ to process, and remove it from S . For each basic path

$$\begin{aligned} L_j &: @ \mu(L_j) \\ S_j; \\ &\vdots \\ S_k; \end{aligned}$$

```

let FORWARDPROPAGATE  $F_{\text{pre}} \mathcal{L} =$ 
   $S := \{L_0\};$ 
   $\mu(L_0) := F_{\text{pre}};$ 
   $\mu(L) := \perp$  for  $L \in \mathcal{L} \setminus \{L_0\};$ 
  while  $S \neq \emptyset$  do
    let  $L_j = \text{CHOOSE } S$  in
     $S := S \setminus \{L_j\};$ 
    foreach  $L_k \in \text{succ}(L_j)$  do
      let  $F = \text{sp}(\mu(L_j), S_j; \dots; S_k)$  in
      if  $F \not\Rightarrow \mu(L_k)$ 
      then  $\mu(L_k) := \mu(L_k) \vee F;$ 
       $S := S \cup \{L_k\};$ 
    done;
  done;
 $\mu$ 

```

Fig. 12.2. FORWARDPROPAGATE

$L_k : @ \mu(L_k)$

starting at L_j , compute if

$$\text{sp}(\mu(L_j), S_j; \dots; S_k) \Rightarrow \mu(L_k) . \quad (12.4)$$

If the implication holds, then $\text{sp}(\mu(L_j), S_j; \dots; S_k)$ does not represent any states that are not already represented by $\mu(L_k)$, so do nothing. Otherwise, set

$$\mu(L_k) := \mu(L_k) \vee \text{sp}(\mu(L_j), S_j; \dots; S_k) .$$

In other words, assign $\mu(L_k)$ to be the union of the set of states represented by $\text{sp}(\mu(L_j), S_j; \dots; S_k)$ with the set of states currently represented by $\mu(L_k)$. Additionally, add L_k to S so that future iterations propagate the effect of the new states. For all other locations $L_\ell \in \mathcal{L}$, assign $\mu(L_\ell) := \mu(L_\ell)$.

This procedure is summarized in Figure 12.2. Given a function's precondition F_{pre} and a cutset $\mathcal{L}$ of its locations, FORWARDPROPAGATE returns the inductive map μ if the main loop finishes. In the code, $L_k \in \text{succ}(L_j)$ is a **successor** of L_j if there is a basic path from L_j to L_k .

If at some point S is empty, then implication (12.4) and the policy for adding locations to S guarantees that all VCs (12.3) are valid, so that μ is now an inductive map. For the initialization of μ ensures that

$$F_{\text{pre}} \Rightarrow \mu(L_0) ,$$

while the check in the inner loop and the emptiness of S guarantees that

$$\text{sp}(\mu(L_j), S_j; \dots; S_k) \Rightarrow \mu(L_k) ,$$

which is precisely the verification condition

$$\{\mu(L_j)\}S_j; \dots; S_k\{\mu(L_k)\}.$$

However, there is no guarantee that S ever becomes empty.

Example 12.3. The forward propagation procedure is an algorithm when analyzing hardware circuits: it always terminates. In particular, if a circuit has n Boolean variables, then the set of possible states has cardinality 2^n . Since each state set $\mu(L_i)$ can only grow, it must eventually be the case (within $|\mathcal{L}| \times 2^n$ iterations) that no new states are added to any $\mu(L_i)$, so S eventually becomes empty. ■

12.1.4 Abstraction

Two elements of the forward propagation procedure prohibit it from terminating in the general case. First, as we know from Chapter 2, checking the implication

$$\text{sp}(\mu(L_j), S_j; \dots; S_k) \Rightarrow \mu(L_k)$$

of the inner loop is undecidable for FOL. Second, even if this check were decidable — for example, if we restricted $\mu(L_k)$ to be in a decidable theory or fragment — the **while** loop itself may not terminate, as the following example shows.

Example 12.4. Consider the following loop with integer variables i and n :

```
@L0 : i = 0 ∧ n ≥ 0;
while
  @L1 : ?
  (i < n) {
    i := i + 1;
  }
```

There are two basic paths to consider:

(1)

```
@L0 : i = 0 ∧ n ≥ 0;
@L1 : ?;
```

and

(2)

```
@L1 : ?;
assume i < n;
i := i + 1;
@L1 : ?;
```

To obtain an inductive assertion at location L_1 , apply the procedure of Figure 12.2. Initially,

$$\begin{aligned}\mu(L_0) &\Leftrightarrow i = 0 \wedge n \geq 0 \\ \mu(L_1) &\Leftrightarrow \perp.\end{aligned}$$

Following path **(1)** results in assigning

$$\mu(L_1) := \mu(L_1) \vee (i = 0 \wedge n \geq 0).$$

$\mu(L_1)$ was $\perp$, so that it becomes

$$\mu(L_1) \Leftrightarrow i = 0 \wedge n \geq 0.$$

On the next iteration, following path **(2)** yields

$$\mu(L_1) := \mu(L_1) \vee \text{sp}(\mu(L_1), \text{assume } i < n; i := i + 1).$$

Currently $\mu(L_1) \Leftrightarrow i = 0 \wedge n \geq 0$, so

$$\begin{aligned}F : \text{sp}(i = 0 \wedge n \geq 0, \text{assume } i < n; i := i + 1) \\ &\Leftrightarrow \text{sp}(i < n \wedge i = 0 \wedge n \geq 0, i := i + 1) \\ &\Leftrightarrow \exists i^0. i = i^0 + 1 \wedge i^0 < n \wedge i^0 = 0 \wedge n \geq 0 \\ &\Leftrightarrow i = 1 \wedge n > 0.\end{aligned}$$

Since the implication

$$\underbrace{i = 1 \wedge n > 0}_F \Rightarrow \underbrace{i = 0 \wedge n \geq 0}_{\mu(L_1)}$$

is invalid,

$$\mu(L_1) \Leftrightarrow (i = 0 \wedge n \geq 0) \vee \underbrace{(i = 1 \wedge n > 0)}_F$$

at the end of the iteration.

At the end of the next iteration,

$$\mu(L_1) \Leftrightarrow (i = 0 \wedge n \geq 0) \vee (i = 1 \wedge n > 0) \vee (i = 2 \wedge n > 1),$$

and at the end of the k th iteration,

$$\begin{aligned}\mu(L_1) &\Leftrightarrow (i = 0 \wedge n \geq 0) \vee (i = 1 \wedge n \geq 1) \\ &\quad \vee \cdots \vee (i = k \wedge n \geq k).\end{aligned}$$

Because the implication

$$\begin{aligned}&i = k \wedge n \geq k \\ &\quad \Downarrow \\ &(i = 0 \wedge n \geq 0) \vee (i = 1 \wedge n \geq 1) \vee \cdots \vee (i = k - 1 \wedge n \geq k - 1)\end{aligned}$$

is invalid for any k , the main loop of FORWARDPROPAGATE never finishes.

However, it is obvious that

$$0 \leq i \leq n$$

is an inductive annotation of the loop. ■

The problem is that FORWARDPROPAGATE attempts to compute the exact set of reachable states. A state s is **reachable** if s appears in some computation. Inductive annotations usually over-approximate the set of reachable states: every reachable state s satisfies the annotation, but other unreachable states can also satisfy the annotation. Cleverly over-approximating the reachable state set can force the main loop of FORWARDPROPAGATE to terminate.

The method of **abstract interpretation** addresses these problems. An abstract interpretation makes two compromises to guarantee termination. First, it manipulates an artificially constrained form of state sets so that implication checking is decidable. The constrained form cannot express every possible state set. Rather, the form is chosen to focus on a particular aspect of programs, such as relationships among numerical variables or aliasing in programs with pointers. Second, it introduces a heuristic step, called **widening**, in which it guesses a limit over-approximation to a sequence of state sets. We describe how to construct an abstract interpretation in six steps.

Step 1: Choose an abstract domain D .

In the first step, we choose the form of state sets that the abstract interpretation manipulates. The **abstract domain** D is a syntactic class of Σ -formulae of some theory T ; each member Σ -formula represents a particular set of states (those that satisfy it). In Section 12.2, the **interval abstract domain** D_I consists of conjunctions of $\Sigma_{\mathbb{Q}}$ -literals of the forms

$$c \leq v \quad \text{and} \quad v \leq c ,$$

for constant c and program variable v . In Section 12.3, we fix **Karr's abstract domain** D_K to consist of conjunctions of $\Sigma_{\mathbb{Q}}$ -literals of the form

$$c_0 + c_1x_1 + \cdots + c_nx_n = 0 ,$$

for constants $c_0, c_1, \dots, c_n$ and program variables $x_1, \dots, x_n$.

Step 2: Construct a map from FOL formulae to D .

It is useful to glean as much information from existing annotations, such as the function precondition, as possible. Additionally, Step 3 requires interpreting conditions of **assume** statements in D . Hence, define a function

$$\nu_D : \text{FOL} \rightarrow D$$

to map a FOL formula F to element $\nu_D(F)$ of D . For example, the assertion

$$F : i = 0 \wedge n \geq 0$$

at L_0 of the loop of Example 12.4 can be represented in the interval abstract domain by

$$\nu_{D_I}(F) : 0 \leq i \wedge i \leq 0 \wedge 0 \leq n$$

and in Karr's abstract domain by

$$\nu_{D_K}(F) : i = 0$$

with some loss of information.

The map should have the property that for any F ,

$$F \Rightarrow \nu_D(F) .$$

Step 3: Define an abstract **sp**.

Define an **abstract strongest postcondition** operator $\overline{\text{sp}}_D$ for assumption and assignment statements such that

$$\text{sp}(F, S) \Rightarrow \overline{\text{sp}}_D(F, S) \quad \text{and} \quad \overline{\text{sp}}_D(F, S) \in D$$

for statement S and $F \in D$.

Consider first the statement **assume** c . Conjunction is used in computing the strongest postcondition of assumption statements:

$$\text{sp}(F, \text{assume } c) \Leftrightarrow c \wedge F .$$

Define abstract conjunction $\sqcap_D$ for this purpose such that

$$F_1 \wedge F_2 \Rightarrow F_1 \sqcap_D F_2 \quad \text{and} \quad F_1 \sqcap_D F_2 \in D$$

for $F_1, F_2 \in D$. Then for $F \in D$,

$$\overline{\text{sp}}_D(F, \text{assume } c) \Leftrightarrow \nu_D(c) \sqcap_D F .$$

Often, the abstract domain D consists of conjunctions of literals of some form; in this case, $\sqcap_D$ is just the usual conjunction $\wedge$.

Defining $\overline{\text{sp}}$ for assignment statements is more complex. For suppose that we use the standard definition

$$\text{sp}(F[v], v := e[v]) \Leftrightarrow \underbrace{\exists v^0. v = e[v^0] \wedge F[v^0]}_G ,$$

which requires existential quantification. Then, later, when we compute the validity of an implication

$$G \Rightarrow \mu(L) ,$$

$\mu(L)$ can contain existential quantification, resulting in a quantifier alternation. However, most decision procedures apply only to quantifier-free formulae, including the procedures of Chapters 8-10. Therefore, introducing existential quantification in $\overline{\text{sp}}$ is undesirable.

We look at specific instances in Sections 12.2 and 12.3.

Step 4: Define abstract disjunction.

Disjunction is applied in FORWARDPROPAGATE to grow the formulae $\mu(L)$ representing the state sets at locations $L \in \mathcal{L}$. Define abstract disjunction $\sqcup_D$ for this purpose, such that

$$F_1 \vee F_2 \Rightarrow F_1 \sqcup_D F_2 \quad \text{and} \quad F_1 \sqcup_D F_2 \in D$$

for $F_1, F_2 \in D$.

Unlike conjunction, exact disjunction is usually not represented in the domain D . We examine specific instances in Sections 12.2 and 12.3.

Step 5: Define abstract implication checking.

On each iteration of the inner loop of FORWARDPROPAGATE, validity of the implication

$$\text{sp}(\mu(L_j), S_j; \dots; S_k) \Rightarrow \mu(L_k)$$

is checked to determine whether $\mu(L_k)$ has changed. A proper selection of D ensures that this validity check is decidable. Additionally, some domains admit a simpler check than querying a full decision procedure, as we see in Sections 12.2 and 12.3.

Step 6: Define widening.

While some abstract domains D and abstract strongest postconditions $\overline{\text{sp}}_D$ guarantee that the forward propagation procedure terminates, defining an abstraction is not sufficient to guarantee termination in general; we exhibit such a case in Example 12.10 of Section 12.2. Thus, abstractions that do not guarantee termination are equipped with a widening operator ∇_D .

A **widening operator** ∇_D is a binary function

$$\nabla_D : D \times D \rightarrow D$$

such that

$$F_1 \vee F_2 \Rightarrow F_1 \nabla_D F_2 .$$

```

let ABSTRACTFORWARDPROPAGATE  $F_{\text{pre}} \mathcal{L} =$ 
   $S := \{L_0\};$ 
   $\mu(L_0) := \nu_D(F_{\text{pre}});$ 
   $\mu(L) := \perp$  for  $L \in \mathcal{L} \setminus \{L_0\};$ 
  while  $S \neq \emptyset$  do
    let  $L_j = \text{CHOOSE } S$  in
     $S := S \setminus \{L_j\};$ 
    foreach  $L_k \in \text{succ}(L_j)$  do
      let  $F = \overline{\text{sp}}_D(\mu(L_j), S_j; \dots; S_k)$  in
      if  $F \not\approx \mu(L_k)$ 
      then if WIDEN()
        then  $\mu(L_k) := \mu(L_k) \nabla_D (\mu(L_k) \sqcup_D F);$ 
        else  $\mu(L_k) := \mu(L_k) \sqcup_D F;$ 
         $S := S \cup \{L_k\};$ 
    done;
  done;
   $\mu$ 

```

Fig. 12.3. ABSTRACTFORWARDPROPAGATE

for $F_1, F_2 \in D$. It obeys the following property. Let $F_1, F_2, F_3, \dots$ be an infinite sequence of elements $F_i \in D$ such that for each i ,

$$F_i \Rightarrow F_{i+1} .$$

Define the sequence

$$G_1 = F_1 \quad \text{and} \quad G_{i+1} = G_i \nabla_D F_{i+1} .$$

Notice that $F_i \Rightarrow G_i$. For some i^* and for all $i \geq i^*$,

$$G_i \Leftrightarrow G_{i+1} .$$

That is, the sequence G_i converges even if the sequence F_i does not converge. Intuitively, the widening operator “guesses” an over-approximation to the limit of a sequence of formulae. Of course, a widening operator could always return $\top$; however, better widening operators make more precise guesses.

A proper strategy of applying widening guarantees that the forward propagation procedure terminates.

Abstract Forward Propagation

Figure 12.3 applies the operators defined in these six steps in the **abstract forward propagation algorithm**. Given a function’s precondition F_{pre} and a cutset $\mathcal{L}$ of its locations, ABSTRACTFORWARDPROPAGATE returns the inductive map μ . When WIDEN() determines that widening should be applied,

$\mu(L_k)$ is updated to be the widening of $\mu(L_k)$ and $\mu(L_k) \sqcup_D F$. A proper definition of `WIDEN()` ensures that the procedure terminates. For example, a simple strategy is that after some predetermined number of iterations, `WIDEN()` always evaluates to `true`.

Subsequent sections examine instances of this framework.

12.2 Interval Analysis

In this section, we describe the **interval abstract interpretation**, which finds inductive assertions that are conjunctions of $\Sigma_{\mathbb{Q}}$ -literals of the forms

$$v \leq c \quad \text{and} \quad c \leq v ,$$

for $c \in \mathbb{Q}$. This analysis is useful for generating simple bounds on variables. Although the abstract domain reasons about rational variables, the analysis is applicable to integer variables: the generated invariants hold whether the program variables range over integers or rationals. Our presentation is fairly simple; one could improve some of the operations to provide better invariants.

Example 12.5. In the loop

```
@L0 : i = 0  ∧  n ≥ 0;
while
  @L1 : ?
  (i < n) {
    i := i + 1;
  }
```

the interval analysis discovers the inductive invariant $0 \leq i \wedge 0 \leq n$ at L_1 . ■

Step 1: Construct the domain D_1 .

Fix the domain D_1 to consist of $\perp$, $\top$, and conjunctions of literals of the form

$$c \leq v \quad \text{and} \quad v \leq c ,$$

for $c \in \mathbb{Q}$ and (program) variable v . Throughout our discussion, we assume that formulae are maintained in a **canonical form** according to the following rules:

- Simplify $\perp \wedge F$ to $\perp$.
- Simplify $\top \wedge F$ to F .
- Simplify $c_1 \leq v \wedge c_2 \leq v$ to $\max(c_1, c_2) \leq v$.
- Simplify $v \leq c_1 \wedge v \leq c_2$ to $v \leq \min(c_1, c_2)$.
- If $c_1 > c_2$, simplify $c_1 \leq v \wedge v \leq c_2$ to $\perp$.

Maintaining formulae in canonical form guarantees that no two syntactically-unequal formulae in canonical form are equivalent.

While the domain is a syntactic class of $\Sigma_{\mathbb{Q}}$ -formulae, we can represent elements in a way that is suitable for computation. A useful representation is based on **interval arithmetic**. Let $[\ell, u]$ represent the interval from ℓ to u , inclusive. The sum of two intervals is

$$[\ell_1, u_1] + [\ell_2, u_2] = [\ell_1 + \ell_2, u_1 + u_2] .$$

The sum of an interval and a scalar c is found by treating c as the interval $[c, c]$.

For scalar multiplication, compute

$$c[\ell, u] = \begin{cases} [c\ell, cu] & \text{if } c \geq 0 \\ [0, 0] & \text{if } c = 0 \\ [cu, c\ell] & \text{if } c < 0 \end{cases}$$

It is convenient to allow lower and upper bounds to be $-\infty$ or ∞ . Define

$$-\infty + c = -\infty , \quad \infty + c = \infty , \quad c \cdot -\infty = -\infty , \quad \text{and} \quad c \cdot \infty = \infty$$

for $c \geq 0$, and

$$-\infty + c = -\infty , \quad \infty + c = \infty , \quad c \cdot -\infty = \infty , \quad \text{and} \quad c \cdot \infty = -\infty ,$$

for $c < 0$. Also, define the empty interval as

$$[\infty, -\infty] ;$$

multiplying it by scalar c or summing it with other intervals yields the empty interval.

Consider set operations:

- Interval intersection:

$$\begin{aligned} & [\ell_1, u_1] \sqcap [\ell_2, u_2] \\ &= \begin{cases} [\infty, -\infty] & \text{if } \max(\ell_1, \ell_2) > \min(u_1, u_2) \\ [\max(\ell_1, \ell_2), \min(u_1, u_2)] & \text{otherwise} \end{cases} \end{aligned}$$

Intersection is exact: the computed interval represents the set that is the set intersection of the two sets represented by the given intervals.

- Interval union:

$$[\ell_1, u_1] \sqcup [\ell_2, u_2] = [\min(\ell_1, \ell_2), \max(u_1, u_2)]$$

The result is called the **interval hull**. It over-approximates the true union: the computed interval represents a set that may include more elements than the set union of the two sets represented by the given intervals.

For predicates,

$$[\ell_1, u_1] \leq [\ell_2, u_2] \quad \text{if } \ell_1 \leq u_2 ,$$

and

$$[\ell_1, u_1] = [\ell_2, u_2] \quad \text{if } [\ell_1, u_1] \cap [\ell_2, u_2] \neq [\infty, -\infty] .$$

Like interval union, these predicates are over-approximations: if any pair of points in $[\ell_1, u_1]$ and $[\ell_2, u_2]$ satisfy the predicate, then the predicate holds on the intervals.

These definitions are enough to define interval arithmetic over linear arithmetic. For example, let each variable x_i range in the interval $[\ell_i, u_i]$. Then

$$c_0 + c_1x_1 + \cdots + c_nx_n ,$$

for $c_i \in \mathbb{Q}$, ranges in the interval

$$c_0 + c_1[\ell_1, u_1] + \cdots + c_n[\ell_n, u_n] .$$

The correspondence of interval arithmetic with D_1 is the following. $F \in D_1$, which is a conjunction of literals of the form $c \leq v$ and $v \leq c$, asserts that $x_i \in [\ell_i, u_i]$, for variable x_i , where

- $\ell_i = \infty$ and $u_i = -\infty$ if F is the formula $\perp$;
- $\ell_i = d$ if $d \leq x_i$ is a literal of F , and $\ell_i = -\infty$ otherwise;
- $u_i = e$ if $x_i \leq e$ is a literal of F , and $u_i = \infty$ otherwise.

Example 12.6. The formula

$$F : 0 \leq i \wedge i \leq 0 \wedge 0 \leq n$$

is an element of D_1 . It asserts that $i \in [0, 0]$ and $n \in [0, \infty]$. Given F , compute

$$i + 2n = [0, 0] + 2[0, \infty] = [0, \infty] ,$$

and

$$i - 2n = [0, 0] + -2[0, \infty] = [0, 0] + [-\infty, 0] = [-\infty, 0] .$$

■

Step 2: Construct the map ν_{D_1} .

Define the map ν_{D_1} as follows. Consider literal F ; then

$$\nu_{D_1}(F) = \begin{cases} \perp & \text{if } F \Leftrightarrow \perp \\ \frac{b}{a} \leq v \wedge v \leq \frac{b}{a} & \text{if } F \Leftrightarrow av = b \\ v \leq \frac{b}{a} & \text{if } F \Leftrightarrow av \leq b \\ \frac{b}{a} \leq v & \text{if } F \Leftrightarrow b \leq av \\ \top & \text{otherwise} \end{cases}$$

for $a, b \in \mathbb{N}$ such that $a > 0$. Define

$$\nu_{D_1} \left(\bigwedge_i F_i \right) = \bigwedge_i \nu_{D_1}(F_i) .$$

This definition of ν_{D_1} is not as precise as it could be. For consider the case in which we know $G \in D_1$; then it is possible to evaluate the truth-value of, for example,

$$H : c_0 + c_1x_1 + \cdots + c_nx_n \leq 0 ,$$

even when $n > 1$. Define $\nu_{D_1}(H, G)$ to equal the interval evaluation (either $\top$ or $\perp$) of H in the context of G . For all other literals F , define $\nu_{D_1}(F, G) = \nu_{D_1}(F)$. When F has several literals, it is sometimes more precise to compute

$$\nu_{D_1}(F, G \wedge \nu_{D_1}(F)) ,$$

which uses as much information from F as possible.

Example 12.7. Consider

$$F : i = 0 \wedge n \geq 0 \wedge \underbrace{j + 2n \leq 4}_H .$$

Then

$$\nu_{D_1}(F) = 0 \leq i \wedge i \leq 0 \wedge 0 \leq n .$$

Note that $\nu_{D_1}(H) = \top$. Now, given

$$G : 5 \leq j ,$$

compute $G \wedge \nu_{D_1}(F)$:

$$G' : 5 \leq j \wedge 0 \leq i \wedge i \leq 0 \wedge 0 \leq n .$$

Then compute

$$\nu_{D_1}(H, G') = \perp$$

since in the context G' ,

$$j + 2n = [5, \infty] + 2[0, \infty] = [5, \infty] \not\leq [4, 4] .$$

Hence,

$$\nu_{D_1}(F, G \wedge \nu_{D_1}(F)) = \perp .$$

Compare this result to the weaker $\nu_{D_1}(F)$.

Simply computing $\nu_{D_1}(F, G)$ yields

$$\nu_{D_1}(F, G) : 0 \leq i \wedge i \leq 0 \wedge 0 \leq n ,$$

since in the context G ,

$$j + 2n = [5, \infty] + 2[-\infty, \infty] = [-\infty, \infty] \leq [4, 4] .$$

■

Step 3: Define $\overline{\text{sp}}$.

Conjunction is natural in D_1 , so $\sqcap_{D_1}$ is simply normal conjunction $\wedge$.

For **assume** statements, define

$$\overline{\text{sp}}_{D_1}(F, \text{assume } c) \Leftrightarrow \nu_{D_1}(c, F) \wedge F .$$

That is, compute the element of D_1 corresponding to c in the context F , and conjoin it to F .

If c is a conjunction of several literals, the following may compute a more precise formula:

$$\overline{\text{sp}}_{D_1}(F, \text{assume } c) \Leftrightarrow \nu_{D_1}(c, F \wedge \nu_{D_1}(c)) \wedge F .$$

It uses information from c to form a more precise context.

Example 12.8. Consider

$$F : 0 \leq i \wedge i \leq 0 \wedge 0 \leq n .$$

Compute

$$\begin{aligned} \overline{\text{sp}}_{D_1}(F, \text{assume } i \leq n) & \\ \Leftrightarrow \nu_{D_1}(i \leq n, 0 \leq i \wedge i \leq 0 \wedge 0 \leq n \wedge \nu_{D_1}(i \leq n)) \wedge F & \\ \Leftrightarrow \nu_{D_1}(i \leq n, 0 \leq i \wedge i \leq 0 \wedge 0 \leq n) \wedge F & \\ \Leftrightarrow [0, 0] \leq [0, \infty] \wedge F & \\ \Leftrightarrow F & \end{aligned}$$

The context F asserts that $i \in [0, 0]$ and $n \in [0, \infty]$, resulting in the comparison of intervals of the fourth line. ■

Assignment statements are handled via interval arithmetic. Compute $\overline{\text{sp}}_{D_1}(F, v := e)$ as follows. Let G be the conjunction of all the literals of F except those referring to v (at most two refer to v : $c_1 \leq v$ and $v \leq c_2$). If e is a linear expression, let $[\ell, u]$ be the interval evaluation of e in the context F , and

$$\overline{\text{sp}}_{D_1}(F, v := e) \Leftrightarrow \ell \leq v \wedge v \leq u \wedge G ,$$

where $-\infty \leq v$ and $v \leq \infty$ both simplify to $\top$. If e is not a linear expression, then let

$$\overline{\text{sp}}_{D_1}(F, v := e) \Leftrightarrow G .$$

According to G , v can have any value.

Example 12.9. Consider

$$F : 0 \leq i \wedge i \leq 0 \wedge 0 \leq n .$$

To compute $\overline{\text{sp}}_{D_1}(F, i := i + 1)$, let G be the literals of F not involving i ,

$$G : 0 \leq n ,$$

and compute the interval evaluation of $i + 1$ in context F :

$$i + 1 = [0, 0] + [1, 1] = [1, 1] .$$

Then

$$\overline{\text{sp}}_{D_1}(F, i := i + 1) \Leftrightarrow 1 \leq i \wedge i \leq 1 \wedge 0 \leq n .$$

■

Step 4: Define interval disjunction, $\sqcup_{D_1}$.

Define $F_1 \sqcup_{D_1} F_2$ as follows. For each variable x , let F_1 assert that $x \in [\ell_1, u_1]$ and F_2 assert that $x \in [\ell_2, u_2]$. Then $F_1 \sqcup_{D_1} F_2$ asserts that x is in the interval hull of $[\ell_1, u_1]$ and $[\ell_2, u_2]$: $x \in [\ell_1, u_1] \sqcup [\ell_2, u_2]$. The interval hull is defined in **Step 1**.

Step 5: Define interval implication checking.

Validity of the $\Sigma_{\mathbb{Q}}$ -formula $F \rightarrow G$ is decidable for $F, G \in D_1$. However, we can define a more efficient check: let F assert $x_i \in [\ell_i, u_i]$ and G assert $x_i \in [m_i, v_i]$ for each variable x_i . Then $F \Rightarrow G$ iff for each x_i

$$[\ell_i, u_i] \subseteq [m_i, v_i] , \quad \text{i.e., } m_i \leq \ell_i \wedge u_i \leq v_i .$$

Step 6: Define interval widening, ∇_{D_1} .

Interval analysis does not naturally terminate, as the following example illustrates.

Example 12.10. Consider again the loop

```
@L0 : i = 0 ∧ n ≥ 0;
while
  @L1 : ?
  (i < n) {
    i := i + 1;
  }
```

of Example 12.4 with paths

$@L_0 : i = 0 \wedge n \geq 0;$ $@L_1 : ?;$	(1)	
and $@L_1 : ?;$ assume $i < n;$ $i := i + 1;$ $@L_1 : ?;$	(2)	

Initially,

$$\mu(L_0) \Leftrightarrow \nu_{D_1}(i = 0 \wedge n \geq 0) \Leftrightarrow 0 \leq i \wedge i \leq 0 \wedge 0 \leq n .$$

After one iteration, following path **(1)**, $\mu(L_1) \Leftrightarrow \mu(L_0)$.

Following path **(2)** on the next iteration yields

$$\mu(L_1) := \mu(L_1) \sqcup_{D_1} \overline{\text{sp}}_{D_1}(\mu(L_1), \text{assume } i < n; i := i + 1) .$$

Currently $\mu(L_1) \Leftrightarrow 0 \leq i \wedge i \leq 0 \wedge 0 \leq n$, so

$$\begin{aligned} & \text{sp}(0 \leq i \wedge i \leq 0 \wedge 0 \leq n, \text{assume } i < n; i := i + 1) \\ & \Leftrightarrow \text{sp}(0 \leq i \wedge i \leq 0 \wedge 0 \leq n, i := i + 1) \\ & \Leftrightarrow 1 \leq i \wedge i \leq 1 \wedge 0 \leq n \end{aligned}$$

Then the new $\mu(L_1)$ is

$$\begin{aligned} \mu(L_1) &:= \mu(L_1) \sqcup_{D_1} \overline{\text{sp}}_{D_1}(\dots) \\ &\Leftrightarrow (0 \leq i \wedge i \leq 0 \wedge 0 \leq n) \sqcup_{D_1} (1 \leq i \wedge i \leq 1 \wedge 0 \leq n) \\ &\Leftrightarrow 0 \leq i \wedge i \leq 1 \wedge 0 \leq n \end{aligned}$$

Since the implication

$$\underbrace{0 \leq i \wedge i \leq 1 \wedge 0 \leq n}_{\text{new } \mu(L_1)} \Rightarrow \underbrace{0 \leq i \wedge i \leq 0 \wedge 0 \leq n}_{\text{old } \mu(L_1)}$$

is invalid,

$$\mu(L_1) \Leftrightarrow 0 \leq i \wedge i \leq 1 \wedge 0 \leq n$$

at the end of the iteration.

At the end of the k th iteration,

$$\mu(L_1) \Leftrightarrow 0 \leq i \wedge i \leq k \wedge 0 \leq n .$$

It is never the case that the implication

$$0 \leq i \wedge i \leq k \wedge 0 \leq n \Rightarrow 0 \leq i \wedge i \leq k-1 \wedge 0 \leq n$$

is valid, so the main loop of `ABSTRACTFORWARDPROPAGATE` never finishes. ■

Hence, we need to define a widening operator ∇_{D_1} . First, for any $F \in D_1$,

$$F \nabla_{D_1} \perp \Leftrightarrow F \quad \text{and} \quad \perp \nabla_{D_1} F \Leftrightarrow F .$$

Otherwise, let $F, G \in D_1$ be other than $\perp$. For each variable v , suppose that F asserts $v \in [\ell_1, u_1]$ and G asserts $v \in [\ell_2, u_2]$. Then $F \nabla_{D_1} G$ asserts that $v \in [\ell, u]$, where

- $\ell = -\infty$ if $\ell_2 < \ell_1$, and otherwise $\ell = \ell_1$;
- $u = \infty$ if $u_2 > u_1$, and otherwise $u = u_1$.

Intuitively, $F \nabla_{D_1} G$ drops bounds that grow from F to G . Since at most only twice as many finite bounds as variables exist, widening can only be applied a finite number of times before all bounds become stable.

Example 12.11. On the k th iteration (for some small k , say, $k = 3$) of the analysis in Example 12.10, compute

$$\mu(L_1) := \mu(L_1) \nabla_{D_1} (\mu(L_1) \sqcup_{D_1} \overline{\text{sp}}_{D_1}(\mu(L_1), \text{assume } i < n; i := i + 1)) .$$

That is,

$$\begin{aligned} (0 \leq i \wedge i \leq k-1 \wedge 0 \leq n) \nabla_{D_1} (0 \leq i \wedge i \leq k \wedge 0 \leq n) \\ \Leftrightarrow 0 \leq i \wedge 0 \leq n \end{aligned}$$

because the upper bound on i increases from $k-1$ to k . Then

$$\mu(L_1) \Leftrightarrow 0 \leq i \wedge 0 \leq n .$$

While this new $\mu(L_1)$ does not imply the previous one,

$$0 \leq i \wedge 0 \leq n \not\Rightarrow 0 \leq i \wedge i \leq k-1 \wedge 0 \leq n ,$$

one more iteration yields the same $\mu(L_1)$, finishing the analysis. Thus,

$$0 \leq i \wedge 0 \leq n$$

is an inductive assertion at L_1 .

Unfortunately, the interval abstract domain is incapable of representing the more interesting invariant $i \leq n$. ■

12.3 Karr's Analysis

Karr's analysis discovers inductive assertions of the form

$$c_0 + c_1x_1 + \cdots + c_nx_n = 0 ,$$

for $c_i \in \mathbb{Z}$ and program variables x_i . Such assertions are called **affine assertions**. They are useful for tracking the relationship among program variables and loop counters. Karr's analysis can be implemented efficiently, with running time polynomial in the program size.

In this section, we present a simplified version of the analysis that Michael Karr originally proposed. In particular, our analysis ignores guards of loops and **if** statements. We use the notation and concepts from Section 8.2.

Example 12.12. Consider the loop

```
@L0 :  $\top$ ;
i := 0;
j := 0;
k := 0;
while
  @L1 : ?
  (*) {
    k := k + 1;
    if (*) i := i + 1;
    else j := j + 1;
  }
```

The guard $*$ denotes nondeterministic choice: either branch can be taken. Karr's analysis discovers the inductive invariant $i + j = k$ at L_1 . ■

Step 1: Construct the domain D_K .

D_K consists of $\perp$, $\top$, and conjunctions of literals of the form

$$c_0 + c_1x_1 + \cdots + c_nx_n = 0 ,$$

which define affine spaces. An **affine space** is a point, a line, a plane, *etc.* An affine space can be specified by a set of equations

$$A\bar{x} = \bar{b} , \quad \text{abbreviating } \bigwedge_i a_{i1}x_1 + \cdots + a_{in}x_n = b_i ,$$

so that the space is given by points $\bar{v}$ satisfying $A\bar{v} = \bar{b}$; or by a finite set of points $V = \{\bar{v}_1, \dots, \bar{v}_k\}$, so that the space is given by

$$\text{affine}(V) = \left\{ \sum_i \lambda_i \bar{v}_i : \sum_i \lambda_i = 1 \right\} ,$$

that is, the set of affine combinations of vectors in V . An **affine combination** of vectors V is a weighted sum

$$\sum_i \lambda_i \vec{v}_i \quad \text{such that} \quad \sum_i \lambda_i = 1 .$$

For example, the affine combination of two disjoint points is a line passing through both. These two representations are the **constraint representation** and the **vertex representation**, respectively.

Example 12.13. The affine space represented in constraint form by

$$i + j = k$$

has vertex representation

$$\left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\} ,$$

where vectors represent values for $[i \ j \ k]^\top$. Confirm that each vertex satisfies $i + j = k$.

The point

$$\begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + (1) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

is in the affine space because $\lambda_1 + \lambda_2 + \lambda_3 = -1 + 1 + 1 = 1$: it is an affine combination of the vertices. ■

The vertex representation is best suited for the version of Karr's analysis that we present. Recall, though, that the abstract domain is really the set of $\Sigma_{\mathbb{Q}}$ -formulae that are conjunctions of literals of the form

$$c_0 + c_1 x_1 + \cdots + c_n x_n = 0 .$$

The vertex representation of domain elements is convenient for computation.

Step 2: Construct the map ν_{D_K} .

This step is trivial, as this version of Karr's analysis does not use information from annotations or assumption statements. Hence,

$$\nu_{D_K}(F) = \top .$$

Step 3: Define $\overline{\text{sp}}$.

Let

$$\overline{\text{sp}}_{D_K}(F, \text{assume } c) \Leftrightarrow F$$

for any F and c . That is, ignore assumption statements. Ignoring assumption statements is not a terrible loss in precision: at best, only affine guards $c : A\overline{x} = \overline{b}$ could be interpreted within D_K . Such guards are uncommon in practice.

Consider assignment $x_k := e$, where e is an affine expression

$$e_0 + e_1x_1 + \cdots + e_nx_n ,$$

for $e_i \in \mathbb{Q}$. Construct the **affine transformation**

$$A\overline{x} + \overline{b} : \begin{bmatrix} 1 & & & & \\ & 1 & & & \\ & & \vdots & & \\ e_1 & e_2 & \cdots & e_n & \\ & & \vdots & & \\ & & & & 1 \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \vdots \\ e_0 \\ \vdots \\ 0 \end{bmatrix}$$

where the row with $\overline{e}$ is the k th row, corresponding to x_k , and the rest of the matrix is the identity matrix. Abbreviate this transformation with the notation $\llbracket x_k := e \rrbracket$.

Now consider an affine space F represented by a set of vertices V_F . To compute the effect of applying the assignment $x_k := e$, apply $\llbracket x_k := e \rrbracket$:

$$\llbracket x_k := e \rrbracket F = \llbracket x_k := e \rrbracket \text{affine}(V_F) = \text{affine}\{\llbracket x_k := e \rrbracket \overline{v} : \overline{v} \in V_F\} .$$

The transformed affine space is given by applying $\llbracket x_k := e \rrbracket$ to each of the vertices of V_F . Then

$$\overline{\text{sp}}_{D_K}(F, x_k := e) \Leftrightarrow \llbracket x_k := e \rrbracket F$$

for affine expression e .

Example 12.14. Consider the affine space given by vertex representation

$$V = \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} ,$$

for variables $[i \ j \ k]^T$, and assignment $i := 2i + j + 3$. The assignment corresponds to transformation

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} i \\ j \\ k \end{bmatrix} + \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix} .$$

Then

$$\llbracket i := 2i + j + 3 \rrbracket V = \left\{ \begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 3 \\ 0 \\ 0 \end{bmatrix} \right\} = \left\{ \begin{bmatrix} 5 \\ 0 \\ 1 \end{bmatrix} \right\} .$$

■

For assignments $x_k := e$ in which e is not an affine expression, define the **affine hull** $\sqcup_{D_K}$ of two affine spaces F_1 and F_2 with vertex representations V and W , respectively, as

$$F_1 \sqcup_{D_K} F_2 = \left\{ \sum_i \lambda_i \bar{v}_i + \sum_j \mu_j \bar{w}_j : \sum_i \lambda_i + \sum_j \mu_j = 1 \right\} .$$

To implement this definition, simply let $U = V \cup W$ be the vertex representation of $F_1 \sqcup_{D_K} F_2$. Then

$$F_1 \sqcup_{D_K} F_2 = \text{affine}(U) = \left\{ \sum_i \lambda_i \bar{u}_i : \sum_i \lambda_i = 1 \right\} ,$$

which is equivalent to the definition in terms of V and W , as desired. For example, the affine hull of two disjoint points is a line; and the affine hull of a line and a point not on the line is a plane. Clearly, the affine hull vastly overapproximates the union of two affine spaces; however, it is the most precise affine space that includes their union.

Now define

$$\overline{\text{sp}}_{D_K}(F, x_k := e) \Leftrightarrow \llbracket x_k := 0 \rrbracket F \sqcup_{D_K} \llbracket x_k := 1 \rrbracket F$$

when e is not an affine expression. In the new affine space, x_k can have any value. Exercise 12.5 asks the reader to prove this claim.

Example 12.15. Consider affine space given by vertex representation

$$V = \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} ,$$

for variables $[i \ j \ k]^\top$, and non-affine assignment $i := f(i, j, k)$. Compute

$$\begin{aligned}
& \llbracket i := 0 \rrbracket V \sqcup_{D_K} \llbracket i := 1 \rrbracket V \\
&= \left\{ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} \cup \left\{ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \right\} \\
&= \left\{ \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \right\} \cup \left\{ \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} \\
&= \left\{ \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\}
\end{aligned}$$

The final set of vertices represents the set of states in which $j = 0$, $k = 1$, and i is any value. ■

Step 4: Define affine disjunction, $\sqcup_{D_K}$.

The affine hull $\sqcup_{D_K}$, defined in **Step 3**, over-approximates disjunction.

Step 5: Define affine implication checking.

For $F_1, F_2 \in D_K$, whether $F_1 \Rightarrow F_2$ is decidable. But a more efficient test relies on the vertex representations V and W of F_1 and F_2 , respectively. For each $\bar{v} \in V$, check if $\bar{v} \in \text{affine}(W)$: determine if there is a $\bar{\lambda}$ such that

$$\bar{v} = \sum_j \lambda_j \bar{w}_j \quad \text{and} \quad \sum_j \lambda_j = 1 .$$

In other words, determine if there is some affine combination of the elements of W that equals $\bar{v}$. More concisely, determine if there is some $\bar{\lambda}$ such that, for A_W a matrix with columns $\bar{w} \in W$,

$$\begin{bmatrix} A_W \\ \bar{1}^T \end{bmatrix} \bar{\lambda} = \begin{bmatrix} \bar{v} \\ 1 \end{bmatrix} ; \text{ that is, } \bar{1}^T \bar{\lambda} = 1 \quad \text{and} \quad A_W \bar{\lambda} = \bar{v} .$$

This query can be decided efficiently using algorithms for solving linear equations, such as Gaussian elimination.

Then $F_1 \Rightarrow F_2$ iff for all $\bar{v} \in V$, $\bar{v} \in \text{affine}(W)$.

Example 12.16. Consider affine space F_1 given by vertex representation

$$V_1 = \left\{ \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} \right\}$$

and affine space F_2 given by vertex representation

$$V_2 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

Is the implication $F_1 \Rightarrow F_2$ valid? Yes, as

$$\begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + (1) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}$$

and $-1 + 1 + 1 = 1$, so the vertex is in $\text{affine}(V_2)$. ■

Step 6: Define affine widening.

With each growth of a $\mu(L)$, its dimension increases by at least 1 by definition of the affine hull. As each $\mu(L)$ can be at most n -dimensional, for n the number of program variables, the procedure `ABSTRACTFORWARDPROPAGATE` terminates even without the use of widening. Hence, we do not define a widening operator.

Example 12.17. Consider the loop

```
@L0 : ⊤;
i := 0;
j := 0;
k := 0;
while
  @L1 : ?
  (*) {
    k := k + 1;
    if (*) i := i + 1;
    else j := j + 1;
  }
```

which has three basic paths:

(1) —————

```
@L0 : ⊤;
i := 0;
j := 0;
k := 0;
@L1 : ?;
```

which is summarized by transformation

$$\tau_1 : \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} i \\ j \\ k \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix},$$

$$(2) \quad \begin{array}{l} @L_1 : ?; \\ k := k + 1; \\ i := i + 1; \\ @L_1 : ?; \end{array}$$

which is summarized by transformation

$$\tau_2 : I \begin{bmatrix} i \\ j \\ k \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}$$

where, recall, I is the identity matrix, and

$$(3) \quad \begin{array}{l} @L_1 : ?; \\ k := k + 1; \\ j := j + 1; \\ @L_1 : ?; \end{array}$$

which is summarized by transformation

$$\tau_3 : I \begin{bmatrix} i \\ j \\ k \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} .$$

Initially, $\mu(L_0) \Leftrightarrow \top$, so its vertex representation is the set of unit vectors

$$\left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} ,$$

while $\mu(L_1) \Leftrightarrow \perp$, represented by $\emptyset$. Then

$$\mu(L_1) := \mu(L_1) \sqcup_{D_K} \tau_1 \mu(L_0) = \emptyset \cup \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} .$$

For the next iteration, consider the two transitions τ_2 and τ_3 :

$$\begin{aligned} \mu(L_1) := \mu(L_1) \sqcup_{D_K} \tau_2 \mu(L_1) &= \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} \cup \left\{ \tau_2 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \right\} \\ &= \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} . \end{aligned}$$

Next,

$$\begin{aligned}\mu(L_1) &:= \mu(L_1) \sqcup_{D_K} \tau_3 \mu(L_1) = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} \cup \left\{ \tau_3 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \tau_3 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \right\} \\ &= \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}.\end{aligned}$$

The new vertex $[0 \ 1 \ 1]^\top$ is obtained from $\tau_3[0 \ 0 \ 0]^\top$. Note that τ_3 is applied to $[1 \ 0 \ 1]^\top$ as well; however

$$\tau_3 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + (1) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix},$$

and $-1 + 1 + 1 = 1$. Hence, $[1 \ 1 \ 2]^\top$ is redundant.

On the next iteration, we obtain convergence. For

$$\tau_2 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad \tau_2 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (2) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix},$$

and

$$\tau_2 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + (1) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix},$$

so that τ_2 does not modify $\mu(L_1)$. Additionally,

$$\tau_3 \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix}, \quad \tau_3 \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (1) \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} + (1) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix},$$

and

$$\tau_3 \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 2 \\ 2 \end{bmatrix} = (-1) \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + (2) \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix},$$

so that τ_3 does not modify $\mu(L_1)$, either.

Hence, the final vertex representation of $\mu(L_1)$ is

$$V = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 1 \end{bmatrix} \right\}.$$

To obtain the constraint representation of this affine space, solve

$$[V^T \ -\bar{I}] \begin{bmatrix} a_1 \\ \vdots \\ a_n \\ b \end{bmatrix} = \bar{0} ;$$

that is,

$$\begin{bmatrix} 0 & 0 & 0 & -1 \\ 1 & 0 & 1 & -1 \\ 0 & 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ b \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} .$$

In this case, we find the inductive assertion

$$i + j = k$$

at L_1 . ■

12.4 ★Standard Notation and Concepts

Our presentation of abstract interpretation differs markedly from the standard presentation in the literature. To facilitate the reader's foray into the literature, we discuss here the standard notation and concepts and relate it to our presentation. The main idea is to describe an abstract interpretation in terms of a set of operations over two lattices.

Lattices

A **partially ordered set** $(S, \preceq)$, also called a **poset**, is a set S equipped with a **partial order** $\preceq$, which is a binary relation that is

- **reflexive:** $\forall s \in S. s \preceq s$;
- **antisymmetric:** $\forall s_1, s_2. s_1 \preceq s_2 \wedge s_2 \preceq s_1 \rightarrow s_1 = s_2$;
- **transitive:** $\forall s_1, s_2, s_3 \in S. s_1 \preceq s_2 \wedge s_2 \preceq s_3 \rightarrow s_1 \preceq s_3$.

A **lattice** $(S, \sqcup, \sqcap)$ is a set equipped with **join** $\sqcup$ and **meet** $\sqcap$ operators that are

- **commutative:**
 - $\forall s_1, s_2. s_1 \sqcup s_2 = s_2 \sqcup s_1$,
 - $\forall s_1, s_2. s_1 \sqcap s_2 = s_2 \sqcap s_1$;
- **associative:**
 - $\forall s_1, s_2, s_3. s_1 \sqcup (s_2 \sqcup s_3) = (s_1 \sqcup s_2) \sqcup s_3$,
 - $\forall s_1, s_2, s_3. s_1 \sqcap (s_2 \sqcap s_3) = (s_1 \sqcap s_2) \sqcap s_3$;
- **idempotent:**
 - $\forall s. s \sqcup s = s$,

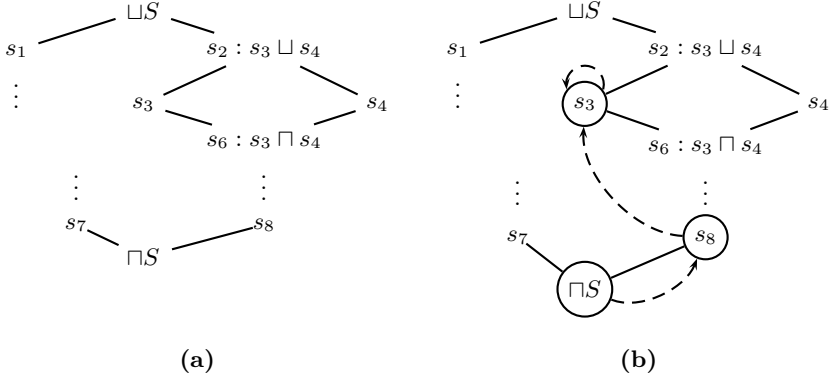


Fig. 12.4. (a) A complete lattice with (b) monotone function f

$$- \quad \forall s. s \sqcap s = s.$$

Additionally, they satisfy the **absorption laws**:

- $\forall s_1, s_2. s_1 \sqcup (s_1 \sqcap s_2) = s_1$;
- $\forall s_1, s_2. s_1 \sqcap (s_1 \sqcup s_2) = s_1$.

One can define a partial order $\preceq$ on S :

$$\forall s_1, s_2. s_1 \preceq s_2 \leftrightarrow s_1 = s_1 \sqcap s_2 ,$$

or equivalently

$$\forall s_1, s_2. s_1 \preceq s_2 \leftrightarrow s_2 = s_1 \sqcup s_2 .$$

A lattice is **complete** if for every subset $S' \subseteq S$ (including infinite subsets), both the join, or **supremum**, $\sqcup S'$ and the meet, or **infimum**, $\sqcap S'$ exist. In particular, a complete lattice has a least element $\sqcap S$ and a greatest element $\sqcup S$. Figure 12.4(a) visualizes a complete lattice.

A function f on elements of a lattice is **monotone** in the lattice if

$$\forall s_1, s_2. s_1 \preceq s_2 \rightarrow f(s_1) \preceq f(s_2) .$$

A **fixpoint** of f is any element $s \in S$ such that $f(s) = s$. The dashed lines of Figure 12.4(b) visualize the iterative application of monotone function f from $\sqcap S$ until it reaches the fixpoint s_3 .

The following theorem is an important classical result for monotone functions on complete lattices.

Theorem 12.18 (Knaster-Tarski Theorem). *The fixpoints of a monotone function on a complete lattice comprise a complete lattice.*

Since complete lattices are nonempty, a fixpoint of a monotone function on a complete lattice always exists. More, the Knaster-Tarski theorem guarantees the existence of the **least fixpoint** and the **greatest fixpoint** of f , which are the least and greatest elements, respectively, of the complete lattice defined by f 's fixpoints.

Abstract Interpretation

An important complete lattice for our purposes is the lattice defined by the sets of a program P 's possible states S : the join is set union, the meet is set intersection, and the partial order is set containment. The greatest element is the set of all states; the least element is the empty set. The lattice is thus represented by $(2^S, \cup, \cap)$, where 2^S is the powerset of S . Call this lattice C_P .

Treat P as a function on S : $P(s)$ is the successor state of s during execution. Define the strongest postcondition on subsets S' of states S and program P :

$$\text{sp}(S', P) \stackrel{\text{def}}{=} \{P(s) : s \in S'\} ,$$

which we abbreviate as $P(S')$. Now define the function

$$F_P(S') \stackrel{\text{def}}{=} S' \cup P(S') ;$$

F_P is a monotone function on C_P . Hence, by the Knaster-Tarski theorem, the least fixpoint of F_P exists on C_P : it is the set of reachable states of P . The lattice C_P and the monotone function F_P define the **concrete semantics** of program P . For this reason, the elements 2^S of C_P comprise the **concrete domain**.

The FORWARDPROPAGATE algorithm of Figure 12.2 can be described as applying F_P to the set of states satisfying the function precondition until the least fixpoint is reached.

However, the least fixpoint of F_P usually cannot be computed in practice (see Section 12.1.4). Therefore, consider another lattice, $A_P : (A, \sqsubseteq, \sqcap)$, whose elements A are “simpler” than those of C_P . Its elements A might be intervals or affine equations, for example, and thus comprise the **abstract domain**. Define two functions

$$\alpha : 2^S \rightarrow A \quad \text{and} \quad \gamma : A \rightarrow 2^S ,$$

the **abstraction function** and the **concretization function**, respectively. These functions should preserve order:

$$\forall S_1, S_2 \subseteq S. S_1 \subseteq S_2 \rightarrow \alpha(S_1) \preceq \alpha(S_2) ,$$

and

$$\forall a_1, a_2 \in A. a_1 \preceq a_2 \rightarrow \gamma(a_1) \subseteq \gamma(a_2) .$$

A monotone function $\overline{F}_P$ on A_P is a **valid abstraction** of F_P if

$$\forall a \in A. F_P(\gamma(a)) \subseteq \gamma(\overline{F}_P(a)) ,$$

that is, if the set of program states represented by abstract element $\overline{F}_P(a)$ is a superset of the application of F_P to the set of states represented by abstract element a . One possible abstraction is given by applying F_P to the concretization of a and then abstracting the result:

$$\overline{F}_P(a) \stackrel{\text{def}}{=} \alpha(F_P(\gamma(a))) .$$

A valid abstraction $\overline{F}_P$ provides a means for approximating the least fixpoint of F_P in C_P : the concretization $\gamma(a)$ of any fixpoint a of $\overline{F}_P$ in A_P is a superset of the least fixpoint of F_P in C_P .

Lattice A_P need not be any better behaved than C_P , though. A well-behaved lattice satisfies the **ascending chain condition**: every nondecreasing sequence of elements eventually converges. Consequently, computing the least fixpoint of a monotone function in such a lattice requires only a finite number of iterations. The lattice defined by the affine spaces of Karr's analysis trivially satisfies this condition: it is of finite height because the number of dimensions is finite. But, for example, the lattice defined by intervals does not satisfy the ascending chain condition. In this case, one must define a widening operator on the abstract domain.

Relating Notation and Concepts

In our presentation, we take as our concrete set not 2^S but rather FOL representations of sets of states. The concrete lattice is thus $(\text{FOL}, \vee, \wedge)$ with partial order $\Rightarrow$. This lattice is not complete: there need not be a finite first-order representation of the conjunction of an infinite number of formulae. But not surprisingly, there need not be a finite representation of a set of infinite cardinality, either, so the completeness of C_P is not of practical value.

Our abstract domains are given by syntactic restrictions on the form of FOL formulae. The abstraction function is ν_D , and the concretization function is just the identity. $\overline{\text{sp}}_D$ is a valid abstraction of sp , and both are monotone in their respective lattices.

12.5 Summary

This chapter describes a methodology for developing algorithms to reason about program correctness. It covers:

- *Invariant generation* in a general setting. The forward propagation algorithm based on the strongest postcondition. The need for abstraction. Issues: decidability and convergence. Abstract interpretations.

- *Interval analysis*, an abstract interpretation with a domain describing intervals. It is appropriate for reasoning about simple bounds on variables.
- *Karr's analysis*, an abstract interpretation with a domain describing affine spaces. It discovers equations among rational or real variables.
- Notation and concepts in the standard presentation of abstract interpretation.

This chapter provides just an introduction to a widely studied area of research.

Bibliographic Remarks

We present a simplified version of the abstract interpretation framework of Cousot and Cousot, who also describe a version of the interval domain that we present [20].

Karr developed his analysis a year before the abstract interpretation framework was presented [46]. For background on linear algebra, see [42]. Our presentation of Karr's analysis is based on that of Müller-Olm and Seidl [63].

Many other domains of abstract interpretation have been studied. The most widely known is the domain of polyhedra, which Cousot and Halbwachs describe in [21]. Exercise 12.4 explores the octagon domain of Miné [61]. As an example of a non-numerical domain, see the work of Sagiv, Reps, and Wilhelm on shape analysis [96].

Exercises

12.1 (wp and sp). Prove the second implication of Example 12.2; that is, prove that

$$F \Rightarrow \text{wp}(\text{sp}(F, S), S) .$$

12.2 (General sp). Compute $\text{sp}(F, \rho_1)$ and $\text{sp}(F, \rho_2)$ for transition relations ρ_1 and ρ_2 of (12.1) and (12.2), respectively. Show that disregarding pc reveals the original definition of sp .

12.3 (General wp and sp). For the general definitions of wp and sp , prove that

$$\text{sp}(\text{wp}(F, S), S) \Rightarrow F \Rightarrow \text{wp}(\text{sp}(F, S), S) .$$

12.4 (Octagon Domain). Design an abstract interpretation for the octagon domain. That is, extend the interval domain to include literals of the form

$$c \leq v_1 + v_2, \quad v_1 + v_2 \leq c, \quad c \leq v_1 - v_2, \quad \text{and} \quad v_1 - v_2 \leq c .$$

Apply it to the loop of Example 12.4. Because i and n are integer variables, the loop guard $i < n$ is equivalent to $i \leq n - 1$.

12.5 (Non-affine assignments). Consider affine spaces F and

$$G : \llbracket x_k := 0 \rrbracket F \sqcup_{D_K} \llbracket x_k := 1 \rrbracket F .$$

Prove the following:

- (a) If $[x_1 \cdots x_k \cdots x_n] \in F$, then $[x_1 \cdots m \cdots x_n] \in G$ for all $m \in \mathbb{R}$.
- (b) If $[x_1 \cdots x_k \cdots x_n] \in G$, then $[x_1 \cdots m \cdots x_n] \in F$ for some $m \in \mathbb{R}$.

Hint: Use the definition of the affine hull.

12.6 (★Analyzing programs).

- (a) Describe how to use `ABSTRACTFORWARDPROPAGATE` to analyze programs with many functions, some of which may be recursive. *Hints:* First, include a location in each function to collect information for the function postcondition. Consider that this information might include function variables other than rv and the parameters, so define an abstract operator `elim` to eliminate these variables. Then recall that function calls in basic paths are replaced by function summaries constructed from the function postconditions. Therefore, `ABSTRACTFORWARDPROPAGATE` need not be modified to handle function calls. But in which order should functions be analyzed? What about recursive functions?
- (b) Describe an instance of this generalization for interval analysis. In particular, define `elim`.
- (c) Describe an instance of this generalization for Karr's analysis. In particular, define `elim`.